

Computer Network Defense: Detection Lab

Table of contents

1	Introduction	1
2	Setup	2
3	Reading the Logs	2
4	Automated Response with fail2ban	3
5	Triggering a Ban	5
6	Optional: Network Intrusion Detection with Suricata	6
7	Cleaning Up	7
8	Wrap-Up Discussion	7

1 Introduction

In the previous labs you prevented unwanted connections with a firewall, isolated software with Docker, and observed network traffic with Wireshark. Prevention will eventually fail, however, so a defender also needs to **detect** suspicious activity and respond to it.

In this lab you will find evidence of failed SSH logins in your system's logs, then set up **fail2ban**, a tool that watches those logs and automatically bans addresses that fail to log in too many times. Optionally, you will also run **Suricata**, a network intrusion detection system, against a packet capture.

Note

All commands in this lab should be run on your **virtual machine**, not your host machine. You will need at least one partner with their own VM on the same network.

2 Setup

This lab uses the SSH server and UFW firewall from the Firewall Lab. Since you reset UFW at the end of that lab, set them up again.

1. Make sure the SSH server is installed and running.

```
sudo apt update
sudo apt install openssh-server
sudo systemctl enable --now ssh
```

2. Allow SSH through the firewall and enable it.

```
sudo ufw allow ssh
sudo ufw enable
sudo ufw status numbered
```

3. Find your IP address and share it with your partner.

```
ip a
```

3 Reading the Logs

Every SSH login attempt, successful or not, is recorded by the system.

1. Have your partner make a few failed login attempts on your machine, using both your real username with a wrong password and a username that does not exist.

```
ssh <your_username>@<your_ip_address>    # type a wrong password
ssh notausers@<your_ip_address>         # any password
```

By default, the SSH client gives three password prompts per connection before giving up.

2. Have your partner log in successfully once, then **exit**.

3. On your machine, view the messages from the SSH server.

```
sudo journalctl -u ssh --since "15 minutes ago"
```

Ubuntu also writes these messages to a plain-text log file.

```
sudo grep sshd /var/log/auth.log | tail -n 30
```

4. Filter for just the failed and successful logins.

```
sudo journalctl -u ssh | grep -E "Failed password|Accepted"
```

5. Count the failed logins from each IP address.

```
sudo journalctl -u ssh | grep "Failed password" | awk '{print $(NF-3)}'  
↵ | sort | uniq -c
```

Each failed login line ends with `from <ip> port <port> ssh2`, so `$(NF-3)` is the fourth field from the end of the line: the IP address. `sort | uniq -c` then counts how many times each address appears.

! Group Discussion

1. Find your partner's failed attempts. What information does each log line give you? Is there anything that distinguishes a wrong password from a username that does not exist?
2. If you were an attacker who had gained root access to this machine, what would you want to do to these logs? How could a defender protect against that?
3. Your SSH server is now reachable by anyone on the network. Who else, other than your partner, appears in the logs, if anyone?

4 Automated Response with fail2ban

Reading logs by hand works for a handful of attempts, but not for an attacker who tries thousands of passwords. **fail2ban** watches the logs for you and, when an address fails too many times within a time window, adds a firewall rule that bans it.

1. Install fail2ban.

```
sudo apt install fail2ban
```

2. Create a local configuration file for fail2ban. Settings in `jail.local` override the defaults and are not overwritten when the package is updated.

```
sudo nano /etc/fail2ban/jail.local
```

Add the following, then save and exit.

```
[sshd]
enabled      = true
maxretry     = 3
findtime     = 10m
bantime      = 5m
banaction    = ufw
```

Settings

- [sshd] is the name of the **jail**: a set of rules for one service.
- maxretry = 3 bans an address after 3 failed attempts...
- findtime = 10m ...if those failures happen within 10 minutes.
- bantime = 5m keeps the address banned for 5 minutes. Real systems use much longer ban times. We use a short one so you are not locked out for the rest of class.
- banaction = ufw tells fail2ban to ban addresses by adding UFW rules, so you can see the bans with the commands from the Firewall Lab.

3. Restart fail2ban so it picks up the new configuration, then confirm it is running.

```
sudo systemctl restart fail2ban
sudo systemctl status fail2ban
```

To exit, press q.

4. Check the status of the sshd jail.

```
sudo fail2ban-client status sshd
```

You should see the number of currently failed attempts and a **Banned IP list** that is empty for now.

Warning

fail2ban never bans the machine it is running on (127.0.0.1), so testing with **ssh localhost** will not trigger a ban. The failed attempts must come from your partner's VM.

5 Triggering a Ban

1. Have your partner attempt to SSH into your machine and type the wrong password until the connection closes. Have them try again if they are not blocked yet.

```
ssh <your_username>@<your_ip_address>
```

After the third failure, their connection attempts should be rejected or should hang, depending on the rule fail2ban adds.

2. On your machine, confirm that your partner's IP address has been banned.

```
sudo fail2ban-client status sshd
```

3. Look at the firewall rule fail2ban added.

```
sudo ufw status numbered
```

You should see a new rule for your partner's IP address at position **1**, above the rule allowing SSH. Recall from the Firewall Lab why the position matters.

4. See what fail2ban recorded about the ban.

```
sudo grep -E "Found|Ban" /var/log/fail2ban.log | tail -n 20
```

5. Unban your partner manually instead of waiting for the ban to expire.

```
sudo fail2ban-client set sshd unbanip <partner_ip_address>
```

Run `sudo ufw status numbered` again to confirm the rule is gone, and have your partner confirm they can log in with the correct password.

6. Switch roles so that every group member has banned and been banned.

! Group Discussion

1. You set `maxretry` to 3. What could go wrong if it were set to 1? What if it were set to 100?
2. Your partner was banned for typing a password wrong, a **false positive**. What would be the effect of a false positive if many users shared one public IP address, e.g. a whole dormitory behind one router?
3. fail2ban only looks at the IP address. How could an attacker keep guessing passwords without ever being banned?

6 Optional: Network Intrusion Detection with Suricata

fail2ban watches the logs on one machine. **Suricata** is a network intrusion detection system: it inspects network traffic and compares it against thousands of signatures of known attacks. Rather than set it up to watch live traffic, you will run it against a packet capture, which is also how analysts review traffic after an incident.

1. Install Suricata and download the free Emerging Threats Open rule set.

```
sudo apt install suricata
sudo suricata-update
```

2. Create a folder for this part of the lab.

```
mkdir ~/suricata-lab
cd ~/suricata-lab
```

3. Start a 30-second capture, as you did in the Wireshark Lab.

```
tshark -i <interface> -a duration:30 -w ids_capture.pcap
```

4. While it is running, in another terminal, request a test page that is designed to trigger a well-known IDS signature. The page is harmless; its content just looks like the output of a successful attack.

```
curl http://testmyids.org/uid/index.html
```

5. Once the capture is complete, run Suricata against it.

```
sudo suricata -r ids_capture.pcap -k none -l .
```

Flags

- `-r` reads packets from a capture file instead of a live interface.
- `-k none` turns off checksum validation. Packets captured on the machine that sent them often have checksums that have not been filled in yet, and Suricata would otherwise ignore them.
- `-l .` writes Suricata's logs to the current folder.

6. View the alerts.

```
cat fast.log
```

You should see an alert similar to `GPL ATTACK_RESPONSE id check returned root`. Suricata also writes a much more detailed record of every alert and connection in JSON format to `eve.json`.

! Group Discussion

1. The test page was harmless, but Suricata alerted anyway. Is this a false positive? What does this tell you about how signature-based detection works?
2. Would Suricata have produced this alert if the page had been requested over HTTPS instead of HTTP? Why or why not? Think back to what you could and could not see in HTTPS traffic in the Wireshark Lab.

7 Cleaning Up

1. Stop fail2ban and prevent it from starting on boot.

```
sudo systemctl disable --now fail2ban
```

2. Reset UFW to remove all rules and disable the firewall. You will be asked to confirm.

```
sudo ufw reset
```

3. If you did the optional section, remove the Suricata lab folder.

```
rm -rf ~/suricata-lab
```

8 Wrap-Up Discussion

! Group Assignment

Before you leave, discuss the **Detection Lab** questions in the *Computer Network Defense: Assignment* document with your group and write down your answers. Your group will submit the answers for all of the labs together, as a single PDF on Canvas, after the final lab.